ПРАКТИЧЕСКАЯ РАБОТА № 1

Разработка перечня артефактов и протоколов проекта

3.2. Описание проекта

Название проекта: UniTask - веб-система управления учебными заданиями университета.

Цель проекта: создать единую цифровую среду, в которой преподаватели размещают задания и учебные материалы, студенты отслеживают сроки, отправляют выполненные работы и получают обратную связь, а администрация управляет курсами и доступом. Система должна уменьшить количество разрозненных сообщений и файлов, обеспечить прозрачность сроков и сохранить историю учебной работы.

Целевая аудитория:

- студенты - просматривают задания, получают уведомления, загружают ответы и следят за оценками;
- преподаватели - создают задания, прикрепляют материалы, проверяют работы и публикуют обратную связь;
- кураторы и руководители образовательных программ - отслеживают выполнение работ и общую учебную активность;
- администраторы университета и техническая поддержка - управляют учетными записями, ролями, курсами и работоспособностью системы.

Ключевые функции системы:

- регистрация и вход пользователей, восстановление доступа и управление профилем;
- разграничение прав для студентов, преподавателей, кураторов и администраторов;
- создание курсов, учебных групп, заданий, сроков сдачи и критериев оценивания;
- публикация учебных материалов и загрузка студентами файлов с выполненными заданиями;
- проверка работ, выставление оценок и предоставление комментариев;
- уведомления о новых заданиях, приближающихся сроках, проверках и изменениях;
- календарь заданий, фильтрация по курсам и просмотр статуса выполнения;
- формирование отчетов по сдаче работ и учебной активности;
- журналирование ключевых действий и резервное копирование данных.

Участники проекта и ответственность:

- заказчик - представитель университета, определяет цели, ограничения и принимает результат;
- руководитель проекта - планирует работы, сроки, ресурсы, риски и коммуникации;

- бизнес-аналитик - собирает требования, описывает сценарии и согласует критерии приемки;
- системный архитектор - определяет структуру решения, интеграции, требования к безопасности и надежности;
- UX/UI-дизайнер - исследует пользовательские сценарии и готовит прототипы интерфейса;
- frontend-разработчик и backend-разработчик - реализуют интерфейс, бизнес-логику и API;
- инженер по тестированию (QA) - готовит тесты, проверяет функции и регистрирует дефекты;
- DevOps-инженер / системный администратор - организует среды, сборку, развертывание, мониторинг и резервное копирование;
- специалист по информационной безопасности - проверяет доступы, риски обработки персональных данных и меры защиты;
- представители пользователей - участвуют в проверке удобства и приемочном тестировании.

3.3. Перечень артефактов проекта

Артефакты сгруппированы по жизненному циклу UniTask: инициирование и анализ, проектирование, разработка, тестирование, внедрение и эксплуатация. Для каждого артефакта указаны назначение, ответственные роли, пользователи, этап создания и место хранения.

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
1	Устав проекта и описание концепции	Документ	Фиксирует цель, границы, ожидаемые результаты, ограничения и критерии успеха.	Руководитель проекта совместно с заказчиком	Вся команда, заказчик	Инициирование	DOCX/PDF в общей папке проекта
2	Матрица участников и ответственности	Таблица / документ	Показывает заинтересованные стороны, полномочия и ответственность за решения и результаты.	Руководитель проекта	Команда, заказчик, кураторы	Инициирование и планирование	XLSX или таблица в Confluence
3	Техническое задание / спецификация требований	Документ	Описывает функциональные и нефункциональные требования, ограничения, роли, безопасность и критерии приемки.	Бизнес-аналитик с заказчиком и архитектором	Заказчик, разработчики, дизайнер, QA	Анализ требований	DOCX/PDF с версиями в хранилище проекта

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
4	Пользовательские истории и критерии приемки	Бэклог / записи трекера	Разбивает требования на проверяемые сценарии для студентов, преподавателей и администраторов.	Бизнес-аналитик, владелец продукта	Разработчики, QA, заказчик	Анализ и планирование; уточняется в ходе проекта	YouTrack/Jira или другой трекер
5	План проекта и структура работ (WBS)	План / таблица	Определяет задачи, зависимости, сроки, контрольные точки и распределение ресурсов.	Руководитель проекта	Вся команда, заказчик	Планирование	XLSX, Gantt или задачи в трекере
6	Описание пользовательских ролей и сценариев	Документ / диаграмма	Фиксирует основные пути выполнения задач каждой ролью и исключительные ситуации.	Бизнес-аналитик, UX/UI-дизайнер	Заказчик, дизайнеры, разработчики, QA	Анализ требований	DOCX, диаграмма или база знаний
7	Прототипы экранов и дизайн-система	Макеты интерфейса	Показывает структуру экранов, состояния форм, навигацию, компоненты и базовые правила доступности.	UX/UI-дизайнер	Заказчик, frontend-разработчик, QA	Проектирование	Figma или экспортированные PDF/PNG

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
8	Архитектурная схема и журнал архитектурных решений	Диаграмма / документ	Описывает компоненты системы, границы модулей, технологии, интеграции и причины важных технических решений.	Системный архитектор совместно с разработчиками	Разработчики, QA, DevOps, специалист по безопасности	Проектирование; обновляется при существенных изменениях	Draw.io и Markdown в Git-репозитории
9	ER-диаграмма и словарь данных	Схема / спецификация	Определяет сущности пользователей, курсов, заданий, сдач, оценок и уведомлений, связи и ограничения данных.	Backend-разработчик и специалист по БД	Backend-разработчик, QA, DevOps, аналитик	Проектирование данных	Диаграмма и описание в репозитории
10	Спецификация API	Техническая спецификация	Определяет адреса методов, форматы запросов и ответов, авторизацию, коды ошибок и версии интерфейсов.	Backend-разработчик под контролем архитектора	Frontend-разработчик, QA, интеграционные участники	Проектирование и разработка	OpenAPI YAML/JSON в Git-репозитории
11	Исходный код приложения	Программный код	Реализует пользовательский интерфейс, бизнес-логику, обработку заданий, оценки и уведомления.	Frontend- и backend-разработчики	Разработчики, QA, DevOps, сопровождающая команда	Разработка	Git-репозиторий с историей версий

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
12	Скрипты миграции и начального заполнения БД	SQL / код	Версионируют изменение структуры базы и позволяют воспроизводимо развернуть нужную схему.	Backend-разработчик / специалист по БД	Разработчики, QA, DevOps, администратор БД	Разработка и внедрение	SQL-файлы в Git-репозитории
13	План тестирования	Документ	Определяет область тестирования, виды проверок, тестовые среды, критерии начала и завершения.	QA-лид	QA, разработчики, руководитель проекта, заказчик	Планирование тестирования	DOCX или Markdown в базе знаний
14	Тест-кейсы и чек-листы	Тестовая документация	Описывают шаги и ожидаемые результаты для входа, прав доступа, создания задания, сдачи файла, оценивания и уведомлений.	QA совместно с аналитиком	QA, разработчики, приемочная группа	Тестирование; готовятся заранее и уточняются по ходу разработки	Система управления тестами или трекер
15	Отчеты о тестировании и реестр дефектов	Отчет / задачи	Фиксируют фактические результаты, найденные ошибки, серьезность, шаги воспроизведения и статус исправления.	QA	Разработчики, руководитель проекта, заказчик	Тестирование и приемка	Задачи в трекере, отчет в PDF/HTML

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
16	Оценка рисков безопасности и защиты данных	Документ / реестр рисков	Определяет риски доступа к персональным данным, утечки учебных работ, нарушения прав и способы снижения рисков.	Специалист по безопасности совместно с архитектором	Заказчик, разработчики, DevOps, администратор	Анализ, проектирование и проверка перед выпуском	DOCX/Markdown с ограниченным доступом
17	Конфигурация CI/CD и автоматизированных проверок	Конфигурационный код	Автоматизирует сборку, запуск тестов, анализ качества и подготовку версий к развертыванию.	DevOps-инженер совместно с разработчиками	Разработчики, QA, DevOps	Разработка и внедрение	YAML/скрипты в Git-репозитории
18	Конфигурация сред и инфраструктуры	Конфигурационные файлы	Описывает повторяемую настройку тестовой и рабочей среды; секреты не хранятся в открытом репозитории.	DevOps-инженер / системный администратор	DevOps, администратор, разработчики	Разработка и внедрение	Docker/Compose, IaC и пример .env в репозитории
19	Инструкция по развертыванию и откату версии	Эксплуатационная инструкция	Содержит порядок выпуска, предварительные проверки, действия при сбое и безопасный откат на предыдущую версию.	DevOps-инженер	Администраторы, дежурная команда, руководитель выпуска	Внедрение и эксплуатация	Markdown/PDF в базе знаний

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
20	Руководство пользователя	Пользовательская документация	Объясняет студентам и преподавателям работу с курсами, заданиями, файлами, оценками и уведомлениями.	Бизнес-аналитик / технический писатель при проверке команды	Студенты, преподаватели, служба поддержки	Подготовка к выпуску; обновление после релизов	PDF/HTML на портале поддержки
21	Руководство администратора и регламент восстановления	Эксплуатационная документация	Описывает управление учетными записями, мониторинг, резервное копирование, восстановление и действия при инцидентах.	DevOps и системный администратор	Администраторы, техническая поддержка	Внедрение и эксплуатация	Markdown/PDF в защищенной базе знаний
22	Описание релиза и список известных ограничений	Документ выпуска	Перечисляет новые функции, исправления, изменения интерфейсов, известные ограничения и действия пользователей.	Руководитель выпуска совместно с QA и аналитиком	Заказчик, пользователи, поддержка	Подготовка и выпуск релиза	Markdown/HTML или PDF

№	Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
23	Протоколы встреч и журнал принятых решений	Документ / журнал	Сохраняет договоренности, ответственных, сроки и историю важных решений по проекту.	Руководитель проекта или назначенный секретарь	Вся команда, заказчик	На всех этапах	Страница в Confluence/общей базе знаний
24	Журнал обращений и инцидентов	Записи в системе поддержки	Фиксирует проблемы пользователей, приоритет, ход решения, причину и меры профилактики.	Служба поддержки; технические специалисты дополняют данные	Поддержка, администратор, разработчики, руководитель проекта	Эксплуатация и сопровождение	Система заявок / база инцидентов

3.4. Перечень протоколов проекта

Протоколы задают обязательные правила взаимодействия: кто и когда принимает решение, где фиксируется результат и как подтвердить выполнение договоренности.

№	Протокол	Назначение	Участники	Периодичность	Форма фиксации
1	Согласование и изменение требований	Устанавливает порядок подачи запроса на изменение, оценки влияния на сроки и объем, согласования и обновления документации.	Заказчик, руководитель проекта, аналитик, архитектор, затронутые разработчики и QA.	При каждом запросе на изменение; базовые требования утверждаются до разработки.	Запрос и решение в трекере; новая версия спецификации требований.
2	Рабочее совещание проекта	Согласует прогресс, препятствия, ближайшие задачи, риски и решения, требующие участия нескольких ролей.	Руководитель проекта, представители разработки, аналитики, QA, DevOps; заказчик по необходимости.	Еженедельно и дополнительно при критическом блокере.	Протокол встречи в базе знаний с решениями, ответственными и сроками.
3	Проверка кода (code review) и слияние изменений	Задает требования к проверке кода, автоматическим проверкам, одобрению и запрету слияния при критических замечаниях.	Автор изменения, минимум один назначенный рецензент, при необходимости QA или архитектор.	Для каждого запроса на слияние изменений в основную ветку.	Pull request в Git; комментарии, результаты CI и отметка об одобрении.
4	Проведение тестирования и разбор дефектов	Определяет подготовку среды и данных, регистрацию дефектов, приоритеты исправления и повторную проверку.	QA, разработчики, аналитик; руководитель проекта при споре о приоритете.	На каждом цикле тестирования; разбор дефектов не реже двух раз в неделю во время активной проверки.	Тест-менеджмент и трекер дефектов; отчет о тестировании.

№	Протокол	Назначение	Участники	Периодичность	Форма фиксации
5	Выпуск и развертывание версии	Задает критерии готовности релиза, порядок резервного копирования, развертывания, проверки после выпуска и отката.	DevOps, QA, ответственный разработчик, руководитель выпуска, администратор системы.	При каждом релизе или экстренном исправлении.	Чек-лист выпуска, журнал CI/CD, запись о версии и план отката.
6	Приемка результата заказчиком	Определяет проверку реализации по согласованным критериям, регистрацию замечаний и подтверждение приемки.	Представитель заказчика, бизнес-аналитик, QA, руководитель проекта, представители пользователей.	В конце этапа или перед выпуском значимой версии.	Протокол приемочного тестирования, список замечаний и электронное подтверждение в трекере.
7	Согласование API и обмена данными	Устанавливает версионирование API, совместимость изменений, форматы данных, обработку ошибок и уведомление зависимых команд.	Архитектор, backend- и frontend-разработчики, QA, ответственные за интеграции.	При проектировании API и при каждом несовместимом или значимом изменении контракта.	Версионированная OpenAPI-спецификация и результаты контрактных тестов.

№	Протокол	Назначение	Участники	Периодичность	Форма фиксации
8	Обработка инцидентов безопасности и персональных данных	Определяет срочное уведомление ответственных, ограничение доступа, фиксацию событий, оценку последствий и план устранения.	Специалист по безопасности, системный администратор, руководитель проекта, ответственный от заказчика и необходимые технические специалисты.	По событию; критические случаи обрабатываются немедленно.	Запись в защищенном журнале инцидентов, отчет о причинах и корректирующих мерах.
9	Резервное копирование и проверка восстановления	Определяет расписание копирования, место хранения, контроль успешности и периодическую проверку восстановления данных.	DevOps, системный администратор, администратор БД; владелец системы получает отчет.	Автоматически по расписанию; контроль ежедневно, тест восстановления ежемесячно и после существенных изменений.	Логи задач резервного копирования и акт/отчет о тестовом восстановлении.

3.5. Связь артефактов и протоколов

Артефакты описывают результаты работы и сохраняют информацию, а протоколы устанавливают правила их создания, изменения, проверки и утверждения. В проекте UniTask эти элементы образуют единую систему управления качеством и изменениями.

Протокол согласования и изменения требований связан с техническим заданием, пользовательскими историями и планом проекта.

Любое изменение должно быть зарегистрировано, оценено по влиянию на сроки и функции, утверждено уполномоченными участниками и отражено в актуальных версиях требований и бэклога. Без протокола разработчики могут реализовать разные трактовки одного требования.

Протокол code review связан с исходным кодом, архитектурной схемой и оценкой безопасности.

Проверяющий сопоставляет изменение с требованиями, архитектурными решениями и правилами защиты данных. Решение и замечания остаются в pull request. Без такого протокола в основную ветку могут попасть дефекты, небезопасная обработка ролей или несовместимые изменения.

Протокол тестирования связан с планом тестирования, тест-кейсами, критериями приемки и реестром дефектов.

Тест-кейсы задают ожидаемое поведение, протокол определяет порядок выполнения и регистрации результатов, а реестр дефектов сохраняет найденные несоответствия и повторные проверки. Без этой связи невозможно доказать, какие требования проверены и какие проблемы остались.

Протокол выпуска и развертывания связан с конфигурацией CI/CD, инструкцией развертывания, планом отката и описанием релиза.

Перед выпуском команда проверяет прохождение автоматических тестов, наличие резервной копии и готовность отката; после развертывания фиксирует результат. Описание релиза сообщает пользователям, что изменилось. Без протокола повышается риск простоя и потери возможности быстро восстановить предыдущую версию.

Протокол приемки связан с пользовательскими историями, критериями приемки и отчетами о тестировании.

Заказчик принимает функциональность не по устным обещаниям, а по заранее согласованным критериям и фактическим результатам. Замечания записываются как дефекты или новые запросы. Без такого протокола трудно определить, завершена ли работа и соответствует ли результат договоренностям.

Протокол согласования API связан со спецификацией API, исходным кодом и контрактными тестами.

Спецификация служит общим контрактом между frontend и backend. При ее изменении зависимые команды получают уведомление, а контрактные тесты выявляют несовместимость. Без этого протокола интерфейс пользователя может перестать работать после изменения формата ответа сервера.

Протокол резервного копирования связан с эксплуатационным руководством, конфигурацией инфраструктуры и журналом инцидентов.

Регламент задает частоту и проверки копирования, руководство описывает восстановление, а журналы подтверждают выполнение. При инциденте команда использует эти артефакты, чтобы восстановить работоспособность и оценить потери данных.

3.6. Обоснование выбора артефактов и протоколов

1. Какие артефакты обязательны для любого проекта, а какие специфичны для UniTask?

К базовым артефактам, нужным практически любому программному проекту, относятся описание цели и границ проекта, согласованные требования, план работ, исходный код с историей версий, архитектурные решения, тест-план и результаты проверки, инструкции по развертыванию, описание релиза и пользовательская или эксплуатационная документация. Они помогают договориться о результате, организовать совместную разработку, контролировать качество и передать систему на сопровождение.

Специфика UniTask отражена в моделях ролей и прав доступа, ER-диаграмме учебных сущностей, сценариях создания и сдачи заданий, проверке работ и выставлении оценок, календаре сроков, уведомлениях, отчетах по учебной активности, а также в регламентах защиты персональных данных и учебных работ. Спецификация API особенно важна при параллельной работе frontend и backend. Руководства администратора и восстановления нужны из-за хранения учебных данных и необходимости поддерживать доступность системы в период занятий.

2. Какие протоколы критичны для проекта и почему?

Наиболее критичны протокол согласования изменений требований, code review, тестирование и разбор дефектов, выпуск и развертывание, а также обработка инцидентов безопасности. Изменения требований затрагивают расписание и поведение заданий; проверка кода снижает риск ошибок доступа и дефектов; тестирование проверяет корректность сдачи и оценивания; регламент выпуска позволяет контролируемо обновлять сервис; протокол безопасности задает действия при возможной утечке или несанкционированном доступе. Протокол приемки также принципиален: он формально подтверждает, что результат соответствует критериям заказчика.

3. Что произойдет, если артефакт или протокол будет отсутствовать?

Если отсутствует спецификация требований, участники могут по-разному понимать права пользователей, логику дедлайнов и правила оценивания; это приведет к переделкам и конфликтам при приемке. Без тест-кейсов и отчетов команда не получит систематического подтверждения, что студент не может просматривать чужие работы, а преподаватель видит только разрешенные ему курсы. Без спецификации API frontend и backend могут использовать разные названия полей или форматы дат, что вызовет ошибки интеграции.

Отсутствие инструкции по развертыванию и протокола выпуска увеличивает вероятность простоя: команда может забыть миграцию БД, не проверить критические сценарии или не суметь откатить релиз. Если не определены резервное копирование и восстановление, повреждение базы может привести к потере отправленных работ и выставленных оценок. Без журнала решений и протокола изменений невозможно восстановить, кто и почему согласовал изменение, а без протокола безопасности участники могут поздно сообщить об инциденте и несогласованно реагировать на него.

Вывод

Для UniTask сформирован перечень из 24 артефактов и 9 протоколов, охватывающих инициирование, анализ требований, проектирование, разработку, тестирование, выпуск и эксплуатацию. Артефакты фиксируют проверяемые результаты работы, а протоколы определяют согласованный порядок их создания и использования. Совместное ведение этих материалов снижает риск неоднозначных требований, дефектов, потери данных и несогласованных изменений, а также упрощает приемку и сопровождение системы.